FireForge

A deep dive from the basics

physics, computation and the distributed system

Written for the FireForge team

November 2026, describing Phases 1–9

Abstract

FireForge is a platform for discovering new materials by computer. It predicts how well a crystal would turn heat into electricity, starting from nothing but the positions of its atoms, and does this for many candidates at once on a small cluster of Linux machines. This guide explains the whole thing from first principles: the physics of thermoelectrics, how quantum-mechanical simulation works, what each calculation in the pipeline computes, and how the distributed runtime moves the work and the data between machines. It ends by following one compound, ZrNiSn, through the running system. No background beyond school physics and basic programming is assumed.

Contents

I The big picture 4

1 What FireForge is for 5

1.1 The problem: finding materials by computation . . . 5
1.2 Two halves . . . 5
1.3 The machines in this deployment . . . 5
1.4 How to read this guide . . . 6

II The physics 7

2 Crystals 8

2.1 Atoms in a repeating pattern . . . 8
2.2 Symmetry and space groups . . . 8
2.3 The half-Heusler family . . . 8

3 Thermoelectrics 10

3.1 The Seebeck effect . . . 10
3.2 The figure of merit ZT . . . 10
3.3 Why high ZT is hard . . . 10
3.4 Temperature regimes . . . 10
3.5 What a good candidate needs . . . 11

4 Quantum mechanics on a computer: DFT 12

4.1 Why quantum mechanics . . . 12
4.2 The idea of DFT . . . 12
4.3 The approximations you will see in the records . . . 12
4.4 Quantum ESPRESSO and `pw.x` . . . 13
4.5 Bands and the band gap . . . 13

5 Vibrations: phonons and thermal conductivity 14

5.1 Atoms on springs . . . 14
5.2 Computing force constants: finite displacements . . . 14
5.3 Anharmonicity and the Grüneisen parameter . . . 14
5.4 Tier 1: the Slack model . . . 15
5.5 Tier 2: phono3py and the Boltzmann equation . . . 15

6 Electrons in motion: transport 16

6.1 The Boltzmann picture . . . 16
6.2 The constant relaxation time approximation . . . 16
6.3 BoltzTraP2 . . . 16
6.4 Putting ZT together . . . 16

7 Stability **17**
7.1 Thermodynamic: formation energy and the convex hull . . . 17
7.2 Dynamic stability . . . 17
7.3 Mechanical stability: elastic constants . . . 17

8 The pipeline and its validation **18**
8.1 One material, end to end . . . 18
8.2 Screening . . . 18
8.3 Validation against experiment . . . 18

III The distributed system **20**

9 Why a custom runtime **21**

10 Phase 1: running jobs reliably on one machine **22**
10.1 The daemon and its parts . . . 22
10.2 Jobs and their life cycle . . . 22
10.3 Running an attempt safely . . . 22
10.4 Content-addressed artifacts . . . 23
10.5 Crash recovery . . . 23
10.6 The wire format . . . 23
10.7 Workers and the Python SDK . . . 23

11 Phase 2: the control plane **24**
11.1 `tdrcp` and PostgreSQL . . . 24
11.2 Identity: a private certificate authority . . . 24
11.3 Registration, heartbeats and health . . . 24
11.4 Campaigns . . . 24

12 Phase 3: peer networking **25**

13 Phase 4: distributed execution **26**
13.1 A central queue, pulled by nodes . . . 26
13.2 Leases and fencing . . . 26
13.3 The node side . . . 26

14 Phase 5: moving data between machines **27**

15 Phase 6: workflows **28**
15.1 DAGs . . . 28
15.2 Dynamic fan-out . . . 28
15.3 Checkpoints . . . 28
15.4 Ranked placement . . . 28

16 Phase 7: one job across several machines **29**

17 Phase 8: wiring in the science **30**
17.1 Records: from a worker to the database . . . 30
17.2 The materials database . . . 30
17.3 The science worker and workflow builders . . . 30

18 Phase 9: the web dashboard **31**

IV Putting it together **32**

19 Following ZrNiSn through the live system **33**
19.1 Submission . . . 33
19.2 A relaxation, step by step . . . 33
19.3 What happens next . . . 33
19.4 Watching it . . . 34

20 Where things live **35**
20.1 Repository . . . 35
20.2 On the machines . . . 35
20.3 Development rules . . . 35

A Glossary **36**

B Units used in records **38**

Part I

The big picture

CHAPTER 1

What FireForge is for

1.1 The problem: finding materials by computation

Every technology is limited by its materials. Batteries are limited by electrodes, solar cells by absorbers, engines by alloys that survive heat. Finding a better material by experiment is slow: you have to guess a composition, synthesise it, grow a clean sample and measure it, which can take months for one compound. The number of possible compounds is astronomically larger than anyone could make.

Since the 1990s, physics and computing have reached the point where many material properties can be *predicted* from quantum mechanics before anyone makes the material. The input is only which atoms are present and where they sit. The output is energies, electronic band structures, vibrations, conductivities and stabilities. Each prediction costs minutes to days of computer time instead of months of lab time, and many can run in parallel. That turns discovery into a search problem: generate candidates, compute their properties, rank them, and send only the best few to an experimental lab.

FireForge is built to run that search. Its first target is **thermoelectric materials**, which convert a temperature difference directly into electric power (and the reverse, for solid-state cooling).

1.2 Two halves

FireForge has two parts that you can understand separately.

The science suite (`science/fireforge_science`). Python code that knows the physics: how to build candidate crystals, drive the quantum-mechanics solver (Quantum ESPRESSO), compute vibrations (Phonopy, phono3py), compute electrical transport (BoltzTraP2), check stability, and combine everything into a thermoelectric figure of merit, ZT. It is organised in modules named CrystalForge, QuantumForge, PhonoForge, TransportForge and StabilityForge.

TDR, the distributed runtime (`runtime/`, `control/`). C++ infrastructure that runs the science code as *jobs* on many machines. It decides which machine runs what, enforces CPU and memory limits, moves result files between machines, retries work when a machine dies, and records every result in a database. It knows nothing about physics.

The science code is written as *workers*: small programs that receive one task ("relax this crystal"), do it, and hand back files and records. TDR treats a worker like any other program. This separation means the physics can change without touching the distributed system, and the runtime could run any other kind of scientific code.

1.3 The machines in this deployment

The **control plane** is the coordinator. It holds the job queue, the workflow state and the materials database, but runs no science itself. The **compute node** does the calculations. More nodes can join at any time: they register, advertise what software they have, and start pulling work. Running jobs never depend on the control plane being up; if it restarts, nodes keep computing and reconnect.

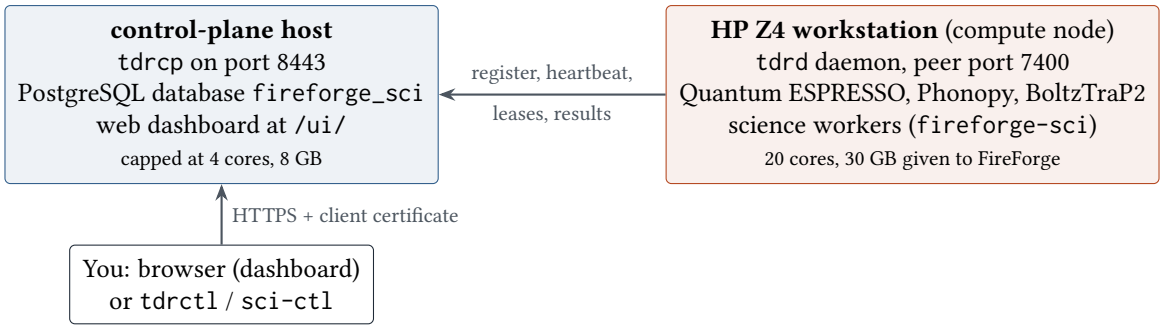


Figure 1.1: The deployment running the Phase 8 validation. Both machines talk over Tailscale (the `100.x.y.z` addresses).

1.4 How to read this guide

Part II builds the physics from the ground up, ending with exactly what each step of the pipeline computes. Part III explains the distributed system, phase by phase, in the order it was built. Part IV follows one compound through the live system and lists the practical commands. A glossary is at the end. You can read Part III before Part II if you are more interested in the software.

Part II

The physics

CHAPTER 2

Crystals

2.1 Atoms in a repeating pattern

A crystal is a solid whose atoms sit in a pattern that repeats in all three directions. Because it repeats, you only need to describe one tile, the **unit cell**, and how it is stacked. The unit cell is a parallelepiped spanned by three **lattice vectors** $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$. Every atom position can be written in **fractional coordinates** (x, y, z), meaning the point $x\,\mathbf{a}_1 + y\,\mathbf{a}_2 + z\,\mathbf{a}_3$, with $0 \le x, y, z < 1$ inside the cell.

That is exactly how FireForge stores a structure: nine numbers for the lattice (three vectors in Å; $1\,\text{Å} = 10^{-10}\,\text{m}$, roughly the size of an atom) and a list of sites, each a chemical species with three fractional coordinates.

> **In FireForge** A real record from the database, ZrNiSn after relaxation (rounded):
>
> ```
> lattice: [0, 3.044, 3.044,   3.044, 0, 3.044,   3.044, 3.044, 0]   (Angstrom)
> sites:   Zr (0, 0, 0)   Ni (0.25, 0.25, 0.25)   Sn (0.5, 0.5, 0.5)
> spacegroup F-43m (216), prototype half-heusler
> ```
>
> Three atoms in the smallest (primitive) cell. The lattice vectors are the face-centred cubic ones, $\frac{a}{2}(0,1,1)$, $\frac{a}{2}(1,0,1)$, $\frac{a}{2}(1,1,0)$, with $a = 2 \times 3.044 = 6.088\,\text{Å}$ here. The conventional cubic cell with edge a holds four of these.

2.2 Symmetry and space groups

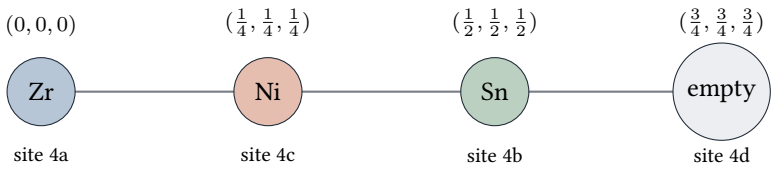
Crystals have symmetry: rotations, mirrors and translations that map the pattern onto itself. Every crystal belongs to one of 230 **space groups**. Symmetry matters for two practical reasons. First, it reduces work: symmetric points need to be computed only once. Second, it gives a robust way to tell whether two structure descriptions are really the same crystal (shifted, rotated, or with atoms listed in another order).

FireForge uses the `spglib` library to find the symmetry and reduce every structure to a standard form. It then hashes that standard form into a **fingerprint** (`fp1:` followed by a SHA-256 hash). Two structures with the same fingerprint are the same crystal, so the database stores it once. A relaxed structure is a new structure that points back to its parent.

2.3 The half-Heusler family

The first campaign studies **half-Heusler** compounds, written XYZ: one transition metal X (e.g. Zr, Hf, Ti, Nb), one late transition metal Y (Ni, Co, Fe), and one main-group element Z (Sn, Sb). They are among the best-studied thermoelectrics at high temperature, are made of relatively cheap elements, and are mechanically tough.

All half-Heuslers share space group $F\bar{4}3m$ (number 216). Picture four interpenetrating face-centred cubic lattices placed along the cube's body diagonal at 0, $\frac{1}{4}$, $\frac{1}{2}$ and $\frac{3}{4}$ of the way. In a half-Heusler three of the four are occupied and one is empty (a full Heusler XY_2Z fills all four):



Which element sits on the special site 4c (between the other two) changes the compound. For three elements there are three distinct **orderings**. Known half-Heuslers put the late transition metal Y on 4c, and FireForge's campaign builder tries all three and keeps the one with lowest energy.

> **The 18-electron rule** Half-Heuslers with 8 or 18 valence electrons per formula unit tend to be semiconductors, which is what a thermoelectric needs. ZrNiSn: Zr 4 + Ni 10 + Sn 4 = 18. NbFeSb: 5 + 8 + 5 = 18. The candidate generator `crystal.halfheusler` uses this count (parameter `electron_counts` = [8, 18]) to discard compositions that are probably metals.

The five compounds in the validation set are ZrNiSn, HfNiSn, TiNiSn, NbFeSb and ZrCoSb. They are all well characterised experimentally, so the predictions can be checked.

CHAPTER 3

Thermoelectrics

3.1 The Seebeck effect

Heat one end of a conductor and cool the other. The mobile charge carriers (electrons or holes) on the hot side have more kinetic energy and diffuse towards the cold side faster than the cold ones diffuse back. Charge piles up at the cold end until the resulting electric field stops the net flow. The voltage this creates is proportional to the temperature difference:

$$V = S\,\Delta T, \tag{3.1}$$

where S is the **Seebeck coefficient**, measured in $\mu\mathrm{V\,K^{-1}}$. For metals S is a few $\mu\mathrm{V\,K^{-1}}$; for good thermoelectric semiconductors it is 150–250 $\mu\mathrm{V\,K^{-1}}$. Its sign tells you the carrier type: negative for electrons (**n-type**), positive for holes (**p-type**). A thermoelectric generator connects n- and p-type legs electrically in series and thermally in parallel.

3.2 The figure of merit ZT

How efficiently a material converts heat to electricity is summarised by one dimensionless number,

$$ZT = \frac{S^2 \sigma\, T}{\kappa_\mathrm{e} + \kappa_\mathrm{l}}, \tag{3.2}$$

with σ the electrical conductivity, T the absolute temperature, and κ the thermal conductivity, split into the part carried by the electrons (κ_e) and the part carried by lattice vibrations (κ_l). The numerator $S^2\sigma$ is the **power factor** (PF, reported in $\mu\mathrm{W\,cm^{-1}\,K^{-2}}$). Commercial materials reach $ZT \approx 1$; ZT above 2 would make waste-heat recovery widely practical.

3.3 Why high ZT is hard

Each quantity pulls against another.

- More carriers raise σ but lower S. Metals conduct well but have tiny S; insulators have huge S but do not conduct. The power factor peaks in heavily doped semiconductors, near 10^{19}–10^{21} carriers per $\mathrm{cm^3}$ (Figure 3.1).
- The electrons that carry charge also carry heat. The Wiedemann–Franz law ties them together: $\kappa_\mathrm{e} = L\sigma T$, with L the Lorenz number. You cannot raise σ without raising κ_e.
- That leaves κ_l as the one quantity you can lower almost independently. Heavy atoms, complex unit cells, strong anharmonicity (non-spring-like bonds) and disorder all scatter vibrations and lower κ_l. This is why FireForge spends so much effort computing κ_l.

3.4 Temperature regimes

ZT depends strongly on temperature, and different applications run at different temperatures. FireForge reports ZT averaged over four windows:

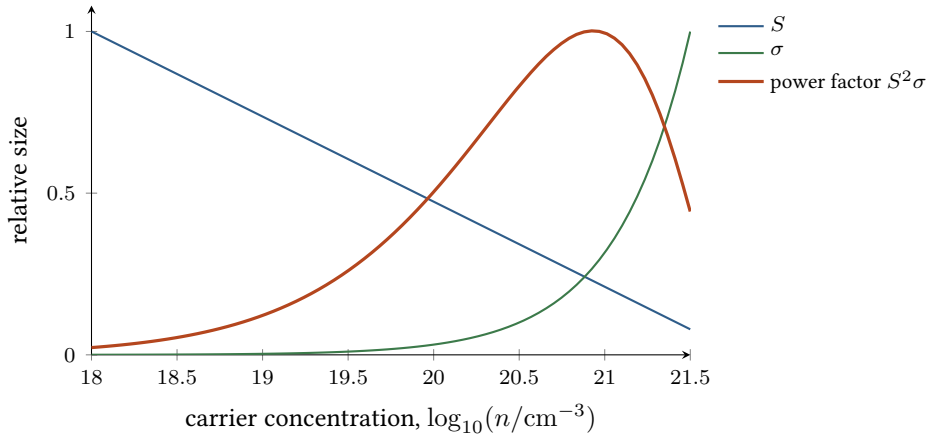


Figure 3.1: Schematic trade-off with doping. S falls and σ rises with carrier concentration, so the power factor has a maximum. Real curves come from the transport calculation (Chapter 6).

Regime	Range	Example use
low	300–500 K	body heat, electronics cooling
intermediate	500–800 K	industrial waste heat, car exhaust
high	800–1100 K	furnaces; half-Heuslers peak here
extreme	above 1100 K	space power (radioisotope generators)

3.5 What a good candidate needs

ZT alone is not enough. A useful material must also be:

- **thermodynamically stable**: it should not decompose into other compounds;
- **dynamically stable**: its atoms should sit at a true energy minimum, not a saddle;
- **mechanically stable**: it should resist shear and compression;
- a **semiconductor** with a band gap, so it can be doped to the right carrier density;
- made of **abundant, affordable, non-toxic** elements.

Each of these becomes a calculation in the pipeline. The rest of Part II explains them.

CHAPTER 4

Quantum mechanics on a computer: DFT

4.1 Why quantum mechanics

The properties above are all determined by how electrons arrange themselves around the nuclei. Electrons obey quantum mechanics: their behaviour is described by a wavefunction satisfying the Schrödinger equation. For a crystal with 10^{23} interacting electrons the exact equation is hopeless. The workhorse approximation that makes it tractable is **density functional theory** (DFT), which earned Walter Kohn a Nobel Prize in 1998 and is now used in tens of thousands of papers a year.

4.2 The idea of DFT

DFT rests on a theorem: the ground-state energy of the electrons is a functional of the *electron density* $n(\mathbf{r})$ alone, a function of three coordinates instead of $3N$. Kohn and Sham turned this into a practical scheme. They replace the real interacting electrons with fictitious non-interacting ones moving in an effective potential chosen so that they have the same density. Each fictitious electron obeys a one-particle equation,

$$\left[-\tfrac{\hbar^2}{2m}\nabla^2 + V_{\text{nuclei}}(\mathbf{r}) + V_{\text{H}}[n](\mathbf{r}) + V_{\text{xc}}[n](\mathbf{r})\right]\psi_i(\mathbf{r}) = \varepsilon_i\,\psi_i(\mathbf{r}), \qquad n(\mathbf{r}) = \sum_{i}^{\text{occupied}} |\psi_i(\mathbf{r})|^2. \quad (4.1)$$

V_{H} is the classical electrostatic repulsion of the density, and V_{xc}, the **exchange–correlation** potential, contains all the hard many-body quantum effects. Its exact form is unknown and must be approximated.

> **The self-consistent loop (SCF)** The potential depends on the density, and the density depends on the solutions. So DFT codes iterate: guess a density, build the potential, solve for the orbitals, compute a new density, mix it with the old one, and repeat until nothing changes (to a tolerance such as 10^{-8} Ry in energy). This is the "SCF" in `qe.scf`.

4.3 The approximations you will see in the records

Every calculation record stores its `method`, which lists the choices below. They matter for interpreting results.

Functional: PBE. FireForge uses the Perdew–Burke–Ernzerhof generalised-gradient approximation for V_{xc}. PBE is the standard for solids. Its known biases: lattice constants come out about 1% too large, and **band gaps come out too small**, often by 30–50%. This is why validation compares predicted gaps with *published PBE* gaps, not with measured ones.

Pseudopotentials: SSSP. Core electrons, close to the nucleus, barely take part in bonding but are expensive to describe. Pseudopotentials replace the nucleus plus core electrons with a smooth effective potential. FireForge uses the curated SSSP 1.3 "PBE efficiency" library, downloaded and checksum-verified by `tools/sci-setup.sh`. Each record lists the files and their MD5 checksums, so results are reproducible.

Plane-wave cutoff. In a periodic crystal it is natural to expand orbitals in plane waves $e^{i\mathbf{G}\cdot\mathbf{r}}$. Precision is controlled by the maximum kinetic energy included, the cutoff (in Ry). SSSP recommends a

cutoff per element; FireForge uses the largest among the elements present, multiplied by 1.25 in the `accurate` preset.

k-points. Electrons in a crystal are labelled by a wave vector **k** in the **Brillouin zone**. Integrals over **k** are done on a grid. FireForge sets the grid by a target spacing (the ASE `kspacing` convention), so denser grids for small cells and coarser ones for big supercells.

4.4 Quantum ESPRESSO and `pw.x`

FireForge uses Quantum ESPRESSO (QE) 6.7, an open-source DFT package; its main program is `pw.x`. FireForge writes an input file, runs `mpirun -np cpus pw.x -in pw.in`, and parses the output files. Three kinds of runs matter:

Relaxation (`qe.relax`). Atoms and the cell shape move downhill in energy until the forces on atoms and the stress on the cell vanish (a "vc-relax", variable-cell relaxation). Output: the equilibrium structure and its total energy. Everything else is computed on this relaxed structure, because properties at the wrong lattice constant are meaningless.

Self-consistent run (`qe.scf`). Solves the SCF loop on a normal k grid and saves the converged density and potential.

Non-self-consistent run (`qe.nscf`). Reuses the saved potential and solves for the orbital energies on a much denser k grid, without iterating. This gives the **band structure** $\varepsilon_n(\mathbf{k})$ in fine detail, which transport needs.

4.5 Bands and the band gap

The orbital energies $\varepsilon_n(\mathbf{k})$ form bands. At zero temperature electrons fill bands up to the Fermi level. In a semiconductor the last filled band (valence band) and first empty band (conduction band) are separated by a **band gap**. The top of the valence band (VBM) and bottom of the conduction band (CBM) can sit at the same **k** (**direct** gap) or at different ones (**indirect**). The `qe.bands` record stores the gap, its character, VBM, CBM and electron count.

> **In FireForge: the first real result** The ZrNiSn screening relaxation on the HP workstation took 591 s on 4 cores and gave $a = 6.150$ Å. The measured value is 6.110 Å, 0.7% smaller, as expected for PBE. Validation allows 2%.

CHAPTER 5

Vibrations: phonons and thermal conductivity

5.1 Atoms on springs

Atoms in a crystal are not frozen; they vibrate about their equilibrium positions. For small displacements the energy rises quadratically, as if neighbouring atoms were joined by springs (the **harmonic approximation**). The "spring constants" are the **second-order force constants** $\Phi_{ij} = \partial^2 E / \partial u_i \partial u_j$. Because the crystal repeats, collective vibrations are waves, and quantised vibrational waves are called **phonons**. Each wave vector $\mathbf{q}$ has $3N$ phonon modes for N atoms per cell: three *acoustic* modes (sound waves) and $3N - 3$ *optical* ones. Phonons carry heat, and that heat flow is κ_l.

5.2 Computing force constants: finite displacements

FireForge computes force constants the direct way, using Phonopy:

1. Build a **supercell**, the unit cell repeated a few times in each direction, so that a displaced atom does not interact with its own periodic copies.
2. Displace one atom by a tiny amount (about 0.01 Å). Symmetry means only a few displacements are needed.
3. Run DFT on each displaced supercell and read the forces on all atoms (`qe.forces`).
4. From force $= -\Phi \times$ displacement, fit Φ. Diagonalising the *dynamical matrix* built from Φ gives the phonon frequencies at any $\mathbf{q}$.

Each displaced supercell is an independent DFT run, so they can run in parallel on different cores or machines. That is why `phonon.harmonic` does not compute anything itself: it *spawns* one `qe.forces` job per supercell plus a `phonon.collect` job to fit the results (the "$\times 10$" boxes in the dashboard's step graph).

> **Imaginary frequencies mean instability** If the energy goes *down* along some displacement, the "spring constant" is negative and the frequency comes out imaginary (shown as negative, in THz). The structure is then not at a minimum and would distort. `phonon.harmonic` records `min_freq_thz` and `dynamically_stable`; unstable candidates are rejected.

5.3 Anharmonicity and the Grüneisen parameter

Perfect springs would let phonons pass through each other forever, giving infinite thermal conductivity. Real bonds are *anharmonic*: stiffness depends on how stretched they are, and this lets phonons scatter off each other. A simple measure is the **Grüneisen parameter** $\gamma = -\partial \ln \omega / \partial \ln V$: how much frequencies shift when the crystal is squeezed. FireForge repeats the phonon calculation at three volumes (strains of -2%, 0 and $+2\%$, the **quasi-harmonic** approach) and averages γ over the acoustic modes. Large γ means strong scattering and low κ_l.

5.4 Tier 1: the Slack model

Computing κ_l exactly is expensive, so FireForge first uses a well-tested formula, the **Slack model**:

$$\kappa_l = A(\gamma)\,\frac{\bar{M}\,\theta_a^3\,\delta\,n^{1/3}}{\gamma^2\,T}, \qquad A(\gamma) = \frac{2.43\times10^{-8}}{1-0.514/\gamma+0.228/\gamma^2}, \tag{5.1}$$

with $\bar{M}$ the average atomic mass (amu), δ^3 the volume per atom (Å^3), n the number of atoms per primitive cell, $\theta_a = \theta_D n^{-1/3}$ the acoustic Debye temperature (K) and $A(\gamma)$ Julian's prefactor. The Debye temperature θ_D comes from the phonon density of states. The model captures the main trends (heavy atoms, low θ_D and large γ give low κ_l, and $\kappa_l \propto 1/T$) but is only accurate to about a factor of two. The `phonon.slack` record stores κ_l at 300, 500, 800 and 1100 K.

5.5 Tier 2: phono3py and the Boltzmann equation

For promising candidates FireForge runs the expensive method. It computes **third-order force constants** (how the force on one atom depends on displacements of two others) by displacing *pairs* of atoms, which takes hundreds of DFT force calculations. With these, phono3py computes every phonon–phonon scattering rate and solves the phonon **Boltzmann transport equation** in the relaxation-time approximation. The result (`phonon.bte`) is typically within 20–30% of experiment for clean crystals.

> **In FireForge** `te.evaluate` spawns this tier automatically when a candidate passes screening (`tier2: auto`). In the validation workflow it is forced for ZrNiSn and turned off for the other four to bound runtime: the third-order calculation is the single most expensive part of the run.

CHAPTER 6

Electrons in motion: transport

6.1 The Boltzmann picture

To get S, σ and κ_e, think of electrons as particles with velocity $\mathbf{v}_n(\mathbf{k}) = \hbar^{-1}\nabla_{\mathbf{k}}\varepsilon_n(\mathbf{k})$, taken directly from the band structure. An electric field or temperature gradient pushes the electron distribution slightly out of equilibrium; scattering (off phonons, impurities, defects) pulls it back over a characteristic **relaxation time** τ. The electronic Boltzmann equation, linearised, gives all three coefficients as integrals over the bands weighted by velocities, τ, and derivatives of the Fermi–Dirac distribution.

6.2 The constant relaxation time approximation

Computing τ from first principles is very expensive. The standard shortcut, used here, is the **constant relaxation time approximation** (CRTA): take τ to be the same for all electrons. Then:

- S does not depend on τ at all. It is a genuine prediction.
- σ and κ_e are proportional to τ. The code computes σ/τ, and a value for τ must be assumed. FireForge uses $\tau = 10^{-14}$ s, typical for these materials.

So ZT values depend on the assumed τ and should be read as estimates for ranking, not absolute predictions. Every record states the τ it assumed (`tau_s`) and flags it in its uncertainty field.

6.3 BoltzTraP2

The transport integrals need band velocities on a very fine k grid. BoltzTraP2 interpolates the DFT bands from the `qe.nscf` grid with smooth Fourier functions and evaluates the integrals. FireForge scans temperatures from 300 to 1200 K and doping levels from 10^{18} to 10^{21} cm^{-3} for both electrons and holes (`transport.boltztrap`). Doping is modelled by shifting the chemical potential, assuming the bands do not change (“rigid band”).

6.4 Putting ZT together

`te.evaluate` combines, at every temperature and doping level, S, σ and κ_e from transport with κ_l from phonons:

$$ZT(T, n) = \frac{S(T, n)^2\, \sigma(T, n)\, T}{\kappa_\mathrm{e}(T, n) + \kappa_\mathrm{l}(T)}.$$

For each temperature it takes the best doping, separately for p- and n-type, and reports the peak ZT, the temperature and doping where it occurs, regime averages, and the curves the dashboard plots.

CHAPTER 7

Stability

7.1 Thermodynamic: formation energy and the convex hull

A compound is only useful if it does not prefer to break into something else. The **formation energy** per atom compares it with its elements in their standard forms:

$$E_f = \frac{E(\mathrm{ZrNiSn}) - E(\mathrm{Zr}) - E(\mathrm{Ni}) - E(\mathrm{Sn})}{3}.$$

Negative means the compound is favoured over the separate elements. That is necessary but not sufficient; it might still prefer a mixture of *other* compounds. The **convex hull** answers this: plot formation energy against composition for every known phase and stretch a lower envelope under them. Phases on the hull are stable. A phase above it decomposes, and its height above the hull, E_{hull} in eV/atom, measures how unstable it is. FireForge accepts $E_{\mathrm{hull}} \leq 0.1$ eV/atom, since some metastable compounds can still be made.

> **In FireForge** This is why the validation workflow relaxes the pure elements too (the `ref.*` and `refrelax.*` steps): every formation energy needs elemental reference energies computed with exactly the same settings. `stability.formation` then builds the hull from the elements plus all compounds in the workflow.

7.2 Dynamic stability

Covered in the phonon chapter: no imaginary phonon frequencies.

7.3 Mechanical stability: elastic constants

Strain the crystal slightly in a few directions, compute the stress with DFT, and fit the **elastic constants** C_{ij} (`qe.elastic`). For a cubic crystal three numbers describe everything: C_{11}, C_{12}, C_{44}. The **Born criteria** for stability are

$$C_{11} - C_{12} > 0, \qquad C_{11} + 2C_{12} > 0, \qquad C_{44} > 0.$$

From the C_{ij} also come the bulk modulus $B = (C_{11} + 2C_{12})/3$, the shear modulus, sound velocities and an independent estimate of the Debye temperature. These are stored in the `stability.elastic` record. The elastic step makes several pw.x runs, so it saves a **checkpoint** after each strain and a retry resumes where it stopped.

CHAPTER 8

The pipeline and its validation

8.1 One material, end to end

Figure 8.1 shows the steps for one compound. Arrows are data dependencies: a step starts as soon as its inputs exist, so the three branches run at the same time on different cores.

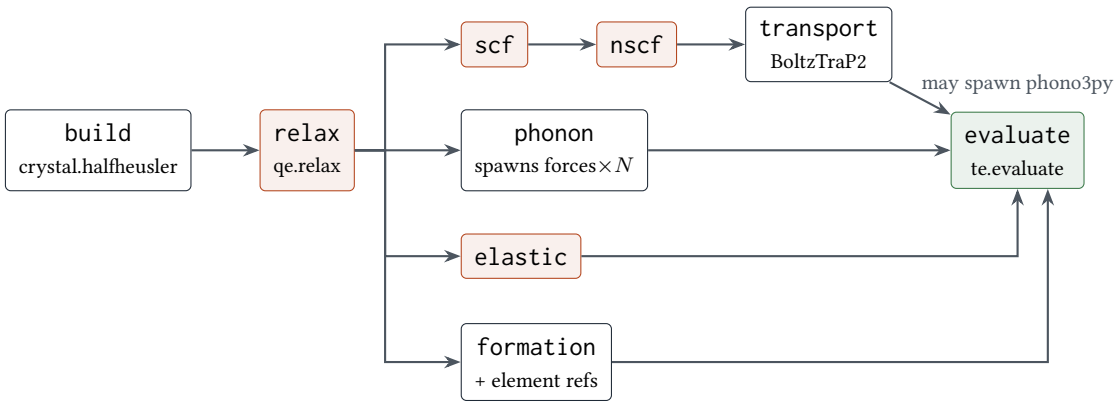


Figure 8.1: The per-material pipeline. Orange steps run DFT with Quantum ESPRESSO.

Step	What it computes	Record kind
`relax`	equilibrium structure and energy	`qe.relax`
`scf`	converged density	`qe.scf`
`nscf`	dense band structure, gap	`qe.bands`
`transport`	S, σ/τ, κ_e vs. T and doping	`transport.crta`
`phonon`	phonons, stability, γ, θ_D, Slack κ_l	`phonon.harmonic`, `phonon.slack`
`elastic`	C_{ij}, Born stability, sound speeds	`stability.elastic`
`formation`	E_f, E_{hull}	`stability.formation`
`evaluate`	$ZT(T)$, screening verdict	`te.evaluate`
phono3py tier	BTE κ_l, re-evaluated ZT	`phonon.bte`
`validate`	comparison with literature	`te.validation`

8.2 Screening

A candidate passes screening when it is dynamically stable, Born stable, $E_{\mathrm{hull}} \leq 0.1$ eV/atom, has a band gap of at least 0.05 eV, and reaches a peak ZT of at least 0.3 (with Slack κ_l) for either carrier type. `te.evaluate` also attaches practical scores (crustal abundance of the scarcest element, cost per kg, and toxicity) from `data/elements.json`.

8.3 Validation against experiment

Before trusting the pipeline on unknown compounds, it is run on five known ones and compared with published values (`data/literature_halfheusler.json`, every value with a DOI). The checks:

- lattice constant within 2% of experiment;

- band gap within 0.2 eV of published PBE values, with the right character (direct or indirect);
- dynamically and mechanically stable, with negative formation energy;
- Slack κ_l within a factor of 3 of experiment; phono3py κ_l for ZrNiSn within a factor of 1.5;
- the ranking of κ_l matches experiment for most adjacent pairs, and the known best p- and n-type materials (NbFeSb, ZrNiSn) come out on top.

The literature values are also stored in the database as `LITERATURE` evidence, so the dashboard shows predicted and measured values side by side. A miss is not hidden: the report must explain it.

Part III

The distributed system

CHAPTER 9

Why a custom runtime

Scientific clusters usually run Slurm, and cloud services run Kubernetes. Both assume dedicated, administered machines. FireForge targets a looser setting: a handful of Linux workstations, possibly in different places, some shared with people doing other work, joined over Tailscale. The requirements were:

- install as an ordinary Linux service (`systemd`), with no cluster administrator;
- strict per-node limits, so a workstation lent to the project keeps a share for its owner;
- survive any machine or the coordinator restarting without losing or duplicating results;
- move large result files directly between machines, not through a central server;
- record where every number came from (provenance).

The result is **TDR**, built in nine phases. Each phase adds one capability on top of the previous ones, and the chapters below follow that order.

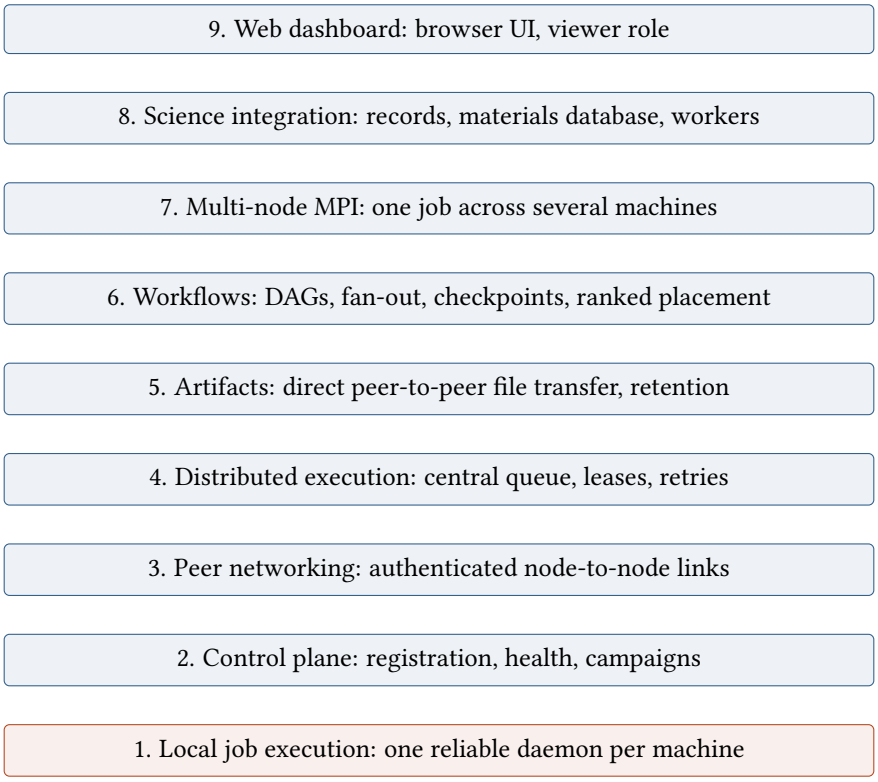


Figure 9.1: The phases, each building on the ones below.

CHAPTER 10

Phase 1: running jobs reliably on one machine

10.1 The daemon and its parts

Each machine runs `tdrd`, a long-lived daemon. You talk to it with `tdrctl` over a Unix socket. Inside, it has a few modules:

JobManager the job state machine: admission, attempts, retries, commit.

Journal an SQLite database recording every state change, so nothing is lost in a crash.

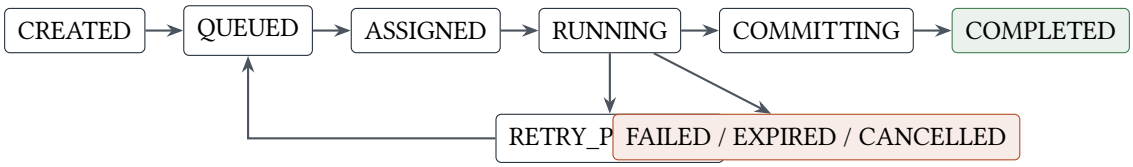
ArtifactStore a content-addressed file store (below).

Supervisor starts each job as an isolated child process with resource limits, and watches it.

The daemon is single-threaded: everything runs on one event loop (Boost.Asio), so no locks are needed, and long work happens in child processes, not in the daemon.

10.2 Jobs and their life cycle

A **job** is one unit of work: a command to run, the resources it needs (cores, memory), required capabilities ("needs QE"), input files, a timeout and a retry policy. Its life:



Each try is an **attempt** with its own ID and its own scratch directory. A failed attempt goes back to the queue if attempts remain. **Admission** is first-in first-out within the node's caps: a job starts only when its cores and memory fit what is left.

10.3 Running an attempt safely

The supervisor launches the job's command as a child process and tracks it with a `pidfd` (a race-free handle to a process). Limits come from **cgroups v2**, the Linux kernel's resource-control mechanism (memory maximum, CPU quota). Where cgroups are not available, it falls back to `setrlimit`, where the memory request becomes a cap on the process's address space.

> **A lesson from the Phase 8 run** That fallback once bit us: spawned science steps that did not request memory got the node's 64 MB default, and numerical libraries failed to even load ("failed to map segment"). Every science step now declares its memory (6 GB for a screening QE step).

A job writes outputs into its attempt directory. When it finishes, the daemon **commits**: it checks the declared outputs exist, moves them into the artifact store and records the result in the journal in one step. Either the whole result is published or none of it is.

10.4 Content-addressed artifacts

Every output file becomes an **artifact**, named by the SHA-256 hash of its content: `artifact://sha256/9f2c...`. Content addressing gives three properties for free: identical files are stored once; a file can never change under its name; and anyone receiving a file can verify it by re-hashing. Inputs to later jobs are referenced by these names.

10.5 Crash recovery

Because every state change is in the journal before it takes effect, a restarted daemon can rebuild its state. It finds attempts that were running, checks whether their processes still exist, and either reattaches or marks them for retry. Half-written outputs live in attempt directories and are discarded; committed ones are in the store.

10.6 The wire format

Programs talk to the daemon (and later, daemons talk to each other) with a fixed binary format: a 48-byte header followed by a Protocol Buffers payload.

Offset	Size	Field	Notes
0	4	magic	`FFTR`
4	2	version	1
6	2	message type	e.g. JOB_SUBMIT, ARTIFACT_REQUEST
8	4	flags	0
12	4	payload length	at most 64 MiB
16	8	message ID	
24	8	correlation ID	the request's ID on a reply
32	16	sender ID	node ID

All integers are big-endian. **Protocol Buffers** (protobuf) is a schema language: message types are defined once in `proto/` files and compiled into C++ and Python code, so both sides agree on the format and new fields can be added without breaking old programs.

10.7 Workers and the Python SDK

A science job is a Python program using the SDK (`sdk/python/fireforge`). The daemon passes the job a socket; the SDK connects, receives a `WorkerTask` (operation name, parameters, input file paths, output directory) and sends back a `WorkerResult`. A worker looks like this:

```
@worker.task("qe.relax")
def relax(ctx):
    structure = load(ctx.input("structure"))
    result = engine.relax(structure, settings)       # runs pw.x
    ctx.save_output("structure", result.structure)
    ctx.record("qe.relax", structure=result.structure,
               properties={"energy_ev_per_atom": ..., "a_angstrom": ...})
```

CHAPTER 11

Phase 2: the control plane

11.1 tdrcp and PostgreSQL

The control plane, `tdrcp`, is a C++ HTTPS server backed by PostgreSQL. In Phase 2 it tracks nodes and campaigns; later phases add the job queue, workflows and the materials database. Its API is REST: JSON over HTTPS (`GET /api/v1/nodes`, `POST /api/v1/workflows`, and so on). The JSON is generated from the same protobuf schemas, so one definition serves C++, Python and the wire.

11.2 Identity: a private certificate authority

Every participant proves who it is with a TLS certificate signed by FireForge's own **certificate authority** (CA). Both sides of each connection present certificates (**mutual TLS**, mTLS). There are no passwords. The certificate's organisational-unit field carries a **role**:

Role	May do
`node`	register, heartbeat, take leases, report job status
`admin`	everything an operator does: submit, cancel, set limits
`viewer`	read everything, change nothing (Phase 9)
`server`	identifies `tdrcp` itself to clients

`tdrctl pki` issues certificates. The science cluster's PKI lives in `~/.local/share/fireforge-sci/pki`.

11.3 Registration, heartbeats and health

When `tdrd` starts, it **registers**: it sends its hardware (cores, memory, GPUs), software **capabilities** (detected ones like `cgroups` plus configured ones such as `sci,qe,phonopy,phono3py,boltztrap2`) and network addresses. It then sends a **heartbeat** every 10 seconds with its current load. The control plane marks a node STALE, then OFFLINE, if heartbeats stop. Heartbeat replies carry the node's **policy**: limits an admin set centrally (for example "use at most 20 cores"), which the node applies live.

11.4 Campaigns

A **campaign** groups related work ("half-Heusler validation, QE"). Pausing a campaign holds all its queued jobs; completing it closes it. Results carry their campaign, so the database can be filtered by it.

CHAPTER 12

Phase 3: peer networking

Nodes need to talk to each other directly, to transfer files and to coordinate multi-node jobs. The control plane publishes a **peer directory** (each node's addresses), and every node keeps a persistent TLS connection to every other online node, a full mesh. A dialler tries the LAN address first and the Tailscale address second, checks the peer's certificate (only `node` certificates are accepted, and the name must match the directory), and measures round-trip time with pings. If a link drops it reconnects with backoff. Reachability is reported to the control plane, which later uses it for placement.

CHAPTER 13

Phase 4: distributed execution

13.1 A central queue, pulled by nodes

Jobs submitted to the control plane go into a central queue in PostgreSQL. Nodes **pull**: whenever a node has spare capacity it asks for work (`POST /api/v1/leases`) and states how many cores and how much memory it has free. The control plane hands out jobs that fit, whose required capabilities the node has, and whose campaign is active. Pulling means a node never receives work it cannot hold, and a busy or dead node simply stops asking.

13.2 Leases and fencing

Each handed-out job is a **lease**, valid for 60 seconds and renewed by every heartbeat. If the node dies, heartbeats stop, the lease expires, the attempt is marked abandoned and the job is queued again for another node.

That raises a classic distributed-systems problem: what if the "dead" node was only cut off, and later comes back and reports a result for a job that has meanwhile been given to someone else? The fix is a **fencing token**: a number that increases with every new attempt. Every status report carries the token of the attempt it belongs to, and the control plane rejects reports whose token is not current. A stale node can never overwrite the real result.

> **Exactly one result** Leases plus fencing plus atomic commit mean each job ends with exactly one published result, even if machines die, networks split, or the control plane restarts mid-job. On restart, the control plane gives nodes one lease period of grace to renew before expiring anything.

13.3 The node side

On the node, a **RemoteExecutor** turns each lease into an ordinary local job (Phase 1), and listens to its state changes. Each change becomes a status update stored in a persistent **outbox** and retried until the control plane acknowledges it, so updates survive daemon restarts and network outages.

CHAPTER 14

Phase 5: moving data between machines

A job's inputs are often another job's outputs, produced on a different machine. Phase 5 lets a node fetch an artifact **directly from a peer** that holds it. The control plane only knows *where* copies are (`artifact_locations`); payloads never pass through it.

- Transfers use a separate TLS data connection to the peer, are chunked, resumable after an interruption, optionally rate-limited, and verified by re-hashing at the end.
- **Placement prefers holders**: a job waits up to 15 seconds (`locality_wait`) for a node that already has its inputs, then goes to anyone, who fetches them.
- **Retention**: each node keeps its store under a disk cap and evicts least-recently-used artifacts, but never the last copy anywhere. Admins can pin artifacts to keep them.

CHAPTER 15

Phase 6: workflows

15.1 DAGs

A scientific result needs many jobs in a particular order. A **workflow** is a directed acyclic graph (DAG) of **steps**. Each step is a job template plus the steps it depends on and which of their outputs it takes as inputs. The workflow engine lives in the control plane: when a step's job finishes, in the same database transaction, it checks which waiting steps are now ready, wires the parents' output artifacts into the children's inputs, and queues their jobs.

Steps have **run rules**: run if all dependencies succeeded (default), if any failed (cleanup), or always (`validate` uses "always", so a report is written even when some compounds fail). A step whose rule is not met is SKIPPED.

15.2 Dynamic fan-out

Sometimes the number of jobs is only known at run time: how many displaced supercells a phonon calculation needs depends on the crystal's symmetry. A running step can therefore **spawn** new steps into its own workflow (`ctx.spawn(...)`). Spawned steps are named `<parent>/<name>`, for example `ZrNiSn.phonon/v0.d001` (volume 0, displacement 1). A step that depends on the spawning step waits for the whole spawned subtree to finish.

15.3 Checkpoints

A long step can save its progress with `ctx.checkpoint(...)`. The checkpoint is stored as an artifact and copied to a nearby peer. If the attempt dies, the retry, on any node, receives the latest checkpoint and resumes instead of starting again.

15.4 Ranked placement

Some jobs care a lot about where they run (big inputs, scarce capabilities). The control plane scores nodes by compute fit, data-transfer cost, queue length, memory headroom and contention, and can **reserve** a job for its best-ranked node for a short window before offering it to the next.

CHAPTER 16

Phase 7: one job across several machines

Very large calculations use MPI, where one program runs as many cooperating processes (**ranks**) that exchange messages. Phase 7 lets one job span several nodes as a **gang**. The control plane reserves capacity on all chosen nodes and **drains** them until every member is free, then leases the whole gang at once. The head node runs `mpirun`, whose remote-launch hook is TDR's own `tdr-launch agent`, so every rank runs inside its node's sandbox and limits, without ssh or Slurm. If any member is lost, the whole gang attempt fails and is retried as a unit from the latest checkpoint.

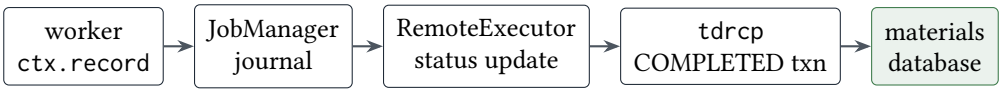
> **In FireForge** The half-Heusler cells have only three atoms, so a single node is plenty and the Phase 8 run does not use gangs. They matter for larger cells later.

CHAPTER 17

Phase 8: wiring in the science

17.1 Records: from a worker to the database

Files are fine for raw data, but you want to *query* results: "all compounds with $ZT > 0.5$". Phase 8 adds **calculation records**. A worker attaches records to its result with `ctx.record(kind, structure=..., properties=..., method=...)`, and they travel with the result:



The control plane stores records in the *same transaction* that marks the job completed, and fencing applies, so a retried attempt can never add duplicates. Records are validated (at most 64 per job and 256 KiB); invalid ones fail the job permanently.

17.2 The materials database

Three tables:

materials one row per reduced formula (ZrNiSn), with its elements.

structures one row per distinct crystal (deduplicated by fingerprint), linked to its material, the job that produced it, and its parent structure.

calculations one row per record: kind, properties (JSON), conditions, method (solver, version, functional, pseudopotentials with checksums, settings), evidence (PREDICTED, LITERATURE or MEASURED), the output artifacts holding raw data, and full provenance: job, attempt, workflow, step, campaign and node.

Any number shown anywhere can be traced back to the exact calculation, settings and machine that produced it.

17.3 The science worker and workflow builders

All science operations live in one worker program, `fireforge_science.ops`, which dispatches on the job's `operation`. Each node has a wrapper called `fireforge-sci` that runs it with the right Python environment, so workflows can say `argv: ["fireforge-sci"]` and run on any science node. Workflows are built in Python (`workflows.py`) and submitted as JSON:

```
python -m fireforge_science submit-validation -o val.json
python -m fireforge_science submit-campaign --x Zr,Hf,Ti --y Ni,Co --z Sn,Sb -o hh.json
tdrctl cp workflow-submit val.json
```

A `toy` engine, a fast analytic stand-in for QE, lets the whole pipeline run in about two minutes for tests. Its results are always labelled `engine: toy` and validation refuses them.

CHAPTER 18

Phase 9: the web dashboard

`tdrcp` also serves a browser UI at `https://<control-plane>:8443/ui/`. The pages are plain JavaScript compiled into the `tdrcp` binary; the browser reads the same REST API as `tdrctl`. Login uses the same certificates: you import a `.p12` file (certificate plus key) into the browser once. A `viewer` certificate sees everything and changes nothing; an `admin` certificate also gets buttons to cancel jobs and workflows, pause campaigns, and set node limits.

Because certificates are sent automatically, like cookies, the server guards against cross-site request forgery: a state-changing request from a browser must come from the dashboard's own origin and carry a JSON body. A strict Content Security Policy forbids any script or style not served by `tdrcp` itself.

Part IV

Putting it together

CHAPTER 19

Following ZrNiSn through the live system

This is the validation workflow `wf-e3988d44d56f0017`, started on 6 November 2026.

19.1 Submission

The workflow was built with `validation_workflow(engine="qe", preset="screening", cpus=4)` and submitted with `sci-ctl workflow-submit`. It has 64 static steps:

- 18 for the nine elements (Co, Fe, Hf, Nb, Ni, Sb, Sn, Ti, Zr): `ref.El` builds the standard crystal of each element, `refrelax.El` relaxes it;
- 9 per compound × 5 compounds = 45: build, relax, formation, scf, nscf, transport, phonon, elastic, evaluate;
- 1 final `validate`.

More steps appear as phonon and phono3py steps spawn their force calculations.

19.2 A relaxation, step by step

1. `ZrNiSn.build` runs first (no dependencies): a light Python job that writes the half-Heusler structure with Ni on 4c. It completes in seconds; its output becomes an artifact on the HP.
2. The engine sees `ZrNiSn.relax` is now ready and queues its job: 4 cores, 6 GB, capabilities `sci`, `qe`, input = the build artifact.
3. The HP's RemoteExecutor, which polls every few seconds while it has room, asks for a lease saying "20 cores free" (say) and receives this job with a fencing token.
4. `tdrd` starts `fireforge-sci` in a fresh attempt directory with a cgroup limited to 4 cores and 6 GB. The worker writes a `pw.in`, runs `mpirun -np 4 pw.x`, and parses the result: three relaxation steps, $a = 6.150$ Å, about 10 minutes.
5. The worker saves outputs (`structure`, `energy`, `log`) and a `qe.relax` record. `tdrd` commits them, and the RemoteExecutor reports COMPLETED with the token, output artifact IDs and the record.
6. In one transaction `tdrcp` marks the job completed, stores the record (creating the structure and linking it to the ZrNiSn material), and queues the four steps waiting for the relaxed structure: scf, phonon, elastic and (once every compound and element has relaxed) formation.

With 20 cores and 4 per QE step, five DFT calculations run at once. The light steps (1–2 cores) fill in the gaps.

19.3 What happens next

`ZrNiSn.phonon` spawns its displaced-supercell force jobs at three volumes plus a collect step; the transport branch runs scf, nscf and BoltzTraP2; elastic runs its strains with checkpoints. When all branches are done, `ZrNiSn.evaluate` computes ZT with Slack κ_l and, because the tier is forced for ZrNiSn, spawns the phono3py steps: several hundred force calculations, the longest part of the whole run.

Finally `validate` compares every compound with the literature and writes `report.md`, `report.json` and `te.validation` records.

19.4 Watching it

- **Dashboard** (`https://<control-plane>:8443/`): Workflows shows the step graph live, Materials shows each compound with predicted-versus-literature checks and ZT curves as results land, Results ranks all evaluated compounds.
- **Command line**:

```
S=~/.local/share/fireforge-sci
$S/sci-ctl workflow wf-e3988d44d56f0017       # step states
$S/sci-ctl nodes                              # node health and load
$S/sci-ctl calcs --kind te.evaluate           # results
$S/sci-ctl material <material-id>             # one compound, latest per kind
```

CHAPTER 20

Where things live

20.1 Repository

Path	Contents
`runtime/`	C++ node runtime: jobs, journal, artifacts, supervisor, peer links, transfer, remote execution, gangs, `tdrd`, `tdrctl`
`control/`	`tdrcp`: database repository, API handlers, server, web dashboard (`control/ui`)
`proto/`	protobuf schemas shared by C++, Python and the wire
`sdk/python/fireforge/`	worker SDK
`science/fireforge_science/`	physics: engines, operations, workflows, literature data
`docs/design/phase*.md`	design of each phase, with implementation notes
`tests/e2e/`	end-to-end tests on real daemons
`tools/`	build, test and setup scripts, all run under the memory cap

20.2 On the machines

Where	What
control plane	service `fireforge-sci-tdrcp`; config and binaries in `~/.local/share/fireforge-sci`; database `fireforge_sci` in `build/pg`
compute node	service `fireforge-sci-tdrd`; config `tdr.conf`; artifacts under `node/artifacts`; code in `~/fireforge` with its own virtual environment
browser access	`~/.local/share/fireforge-sci/browser/`: `viewer.p12`, `admin.p12`, CA certificate

20.3 Development rules

All builds and tests go through `tools/capped`, which puts them in one shared memory-limited cgroup (8 GB, 4 cores) so parallel work never starves the machine:

```
tools/build.sh -B build/dev --test        # C++ build and unit tests
tools/pytest.sh science/tests             # science tests
tools/e2e.sh -k science                   # end-to-end tests
```

APPENDIX A

Glossary

Artifact
: An immutable file named by the SHA-256 hash of its content.

Attempt
: One try at running a job, with its own ID and fencing token.

Band gap
: Energy between the top of the valence band and the bottom of the conduction band.

BoltzTraP2
: Code that computes electronic transport from DFT bands.

Brillouin zone
: The set of distinct electron (or phonon) wave vectors in a crystal.

Campaign
: A named group of related jobs and results.

Capability
: A label a node advertises (`qe`, `phonopy`) and a job can require.

Checkpoint
: Saved progress of a running step, used to resume a retry.

Control plane
: `tdrcp`: the coordinator holding queue, workflows and database.

Convex hull
: Lower envelope of formation energies; phases above it are unstable.

CRTA
: Constant relaxation time approximation: one scattering time for all electrons.

DFT
: Density functional theory: quantum mechanics via the electron density.

Fencing token
: Increasing attempt number that makes stale result reports rejectable.

Fingerprint
: Hash of a symmetry-standardised structure; equal fingerprints mean the same crystal.

Force constants
: Second (or third) derivatives of energy with respect to atomic displacements.

Grüneisen parameter γ
: How phonon frequencies change with volume; measures anharmonicity.

Half-Heusler
: XYZ compound in space group $F\bar{4}3m$ with one empty sublattice.

κ_l, κ_e
: Lattice and electronic thermal conductivity.

Lease
: Time-limited permission for a node to run a job attempt, renewed by heartbeats.

mTLS
: Mutual TLS: both client and server prove identity with certificates.

PBE The exchange–correlation approximation used here; underestimates band gaps.

Phonon
: A quantised lattice vibration; carries heat.

Power factor
: $S^2\sigma$.

Pseudopotential
: Smooth effective potential replacing a nucleus and its core electrons.

Record
: A structured result (kind, properties, method, provenance) stored in the database.

Seebeck coefficient S
: Voltage per kelvin of temperature difference.

Slack model
: Closed-form estimate of κ_{l} from phonon properties.

Spawn
: A running step adding new steps to its own workflow.

Supercell
: Unit cell repeated several times, used for phonon calculations.

TDR FireForge's distributed execution runtime.

Workflow
: A DAG of steps run by the control plane.

ZT Thermoelectric figure of merit, $S^2\sigma T/(\kappa_{\mathrm{e}}+\kappa_{\mathrm{l}})$.

APPENDIX B

Units used in records

Key suffix	Unit	Meaning
`_ev`, `_ev_per_atom`	eV	energy (1 eV ≈ 96.5 kJ/mol)
`_angstrom`	Å	length, 10^{-10} m
`_thz`	THz	phonon frequency
`_k`	K	temperature
`_gpa`	GPa	elastic modulus
`_w_mk`	$\mathrm{W\,m^{-1}K^{-1}}$	thermal conductivity
`_uv_k`	$\mu\mathrm{V\,K^{-1}}$	Seebeck coefficient
`_uw_cmk2`	$\mu\mathrm{W\,cm^{-1}\,K^{-2}}$	power factor
`_cm3`	$\mathrm{cm^{-3}}$	carrier concentration
`_s`	s	time (e.g. relaxation time τ)